

Project: Oracle to PostgreSQL migration

The Problem Statement:

Our Oracle database had served us well, but it came with:

- 1) High licensing & maintenance costs
- 2) Limited elasticity for fluctuating workloads
- 3) Complex scaling in a cloud-first world

We needed an open-source, cost-efficient, and cloud-native solution — without compromising performance.

The Resolution

- 1) We migrated to Azure Database for PostgreSQL – Flexible Server, leveraging:
- 2) Azure Database Migration Service + ora2pg for smooth schema & data transfer
- 3) Optimized storage & compute tiers for workload patterns
- 4) Query tuning & indexing for high performance
- 5) Phased migration strategy to achieve near-zero downtime

The Results

- 1) ~40% cost reduction in database operations
 - 2) Automatic scalability and improved disaster recovery
 - 3) Easier integration with modern analytics & microservices
 - 4) Freedom from vendor lock-in with open-source PostgreSQL.
-

ChatGPT explanation-

"In one of my recent projects, we migrated our existing database from **Oracle** to **Azure PostgreSQL Flexible Server**.

Earlier, our system was running fine on Oracle, but we started facing some issues — mainly the **high licensing cost, difficulty in scaling**, and **less flexibility** for cloud-based workloads. So, we decided to move to **PostgreSQL on Azure**, which is open-source, more cost-efficient, and cloud-native.

For the migration, we used **Azure Database Migration Service** along with **ora2pg**. Ora2pg helped us convert the Oracle schema, procedures, and data into PostgreSQL-compatible format.

We followed a **phased migration approach** to ensure there was **almost zero downtime** — basically moving modules step by step instead of everything at once.

After migration, we did **query tuning and indexing** to make sure the performance was at least as good, if not better, than before.

In the end, we achieved around **40% cost reduction, better scalability**, and easier **integration with analytics and microservices**. Plus, since PostgreSQL is open-source, we're now free from vendor lock-in.

So overall, it was a full cloud modernization project — moving from a costly on-prem/legacy database to a flexible, cloud-optimized PostgreSQL system.”

NOTE:

- **Tools used:** “Azure Database Migration Service, ora2pg, Azure Monitor for performance tracking.”
 - **Challenges faced:** “Main challenge was schema conversion for Oracle-specific functions, but ora2pg handled most of it, and we manually fixed some stored procedures.”
 - **Your role:** “I was responsible for configuring the migration pipeline, verifying data integrity, and optimizing queries post-migration.”
-

Tool Description:

Ora2Pg is a free, open-source tool that helps migrate an Oracle or MySQL database to a PostgreSQL compatible schema. It connects to the source database, scans and extracts its structure and data, and then generates SQL scripts to be loaded into a PostgreSQL database. The tool can handle a variety of database objects, from tables and views to functions, triggers, and packages, and it can export either full schemas or specific selections of objects and data.

How it works

- **Connects to the source database:** Ora2Pg uses Perl DBI to connect to both Oracle and PostgreSQL. It can connect directly to the databases or work with flat files from an Oracle dump.
- **Scans and extracts:** The tool automatically scans the source database to identify and extract its structure (tables, views, indexes, etc.) and data.
- **Generates SQL scripts:** It then produces SQL scripts that contain the schema and data, formatted to be compatible with PostgreSQL.

- **Performs automatic conversion:** Ora2Pg attempts to automatically convert Oracle-specific code like functions, procedures, and packages into their PostgreSQL equivalents (PL/pgSQL), although some manual adjustments may be necessary.

Key features-

- **Handles a wide range of objects:** It can export tables, views, sequences, indexes, and all constraints like primary, foreign, and check constraints.
- **Migrates data:** It supports exporting the full data of tables or a subset defined by a WHERE clause.
- **Converts code:** Includes basic automatic conversion of PL/SQL to PL/pgSQL code.
- **Supports various data types:** It can fully support Oracle's BLOB object by exporting it as BYTEA in PostgreSQL.
- **Migrates security and partitions:** It can export grants/privileges and handles Oracle range/list partitions.
- **Provides assessment:** It can generate a detailed report of the Oracle database content to help with migration cost assessment.